



1-types-io

1. 简介



本节介绍程序的基本组成要素——变量、类型与运算符，以及最基本的输入输出（I/O）操作。

学习目标：学完本节后，你应该能够：

- 使用 C++20 标准声明并初始化变量，掌握统一初始化（uniform initialization）的写法；
- 使用 `std::format` 进行格式化输出（精度、宽度、对齐），使用 `std::cin` 进行输入；
- 理解未定义行为（Undefined Behavior, UB）的概念，并规避最常见的未定义行为；

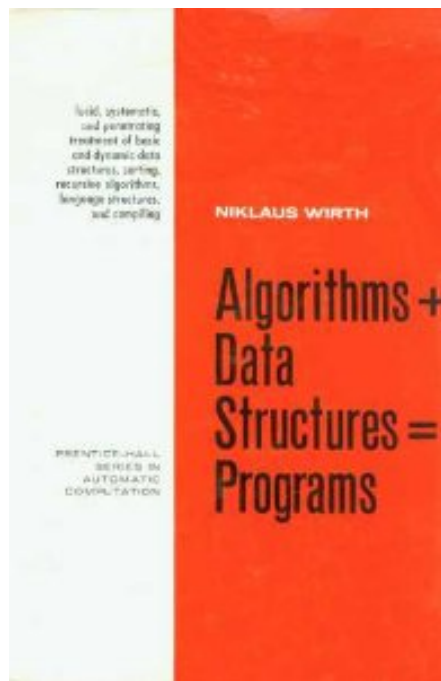
本节重点：

- 变量、类型与表达式；
- `std::cout` 与 `std::format` 的基本用法；`std::cin` 的基本用法；
- 未定义行为的概念与典型场景。

本节难点：

- 未定义行为的成因与规避；
- `std::format` 格式说明（精度、宽度、对齐的格式字符串写法）。

2. 程序 = 数据结构 + 算法



1984 年的图灵奖（[Turing Award](#)）得主 [Niklaus Wirth](#) 写过一本非常有名、以致于人人皆知但没什么人读过的书，书名叫作“算法 + 数据结构 = 程序”。通俗地讲，程序就是对数据（data）进行各种操作（operation）。没有数据，程序毫无意义；仅有数据，却不对数据作任何处理，这样的程序也毫无用处。在程序设计语言中，我们用变量（variable）表示数据，用运算符（operator）表示操作。

另外，程序通常还需要与外界进行交互，包括从外界获取数据以及将处理过的数据向外界进行展示。这涉及到最基本的输入（input）、输出（output）操作。

本节将介绍变量、类型、运算符与输入输出（I/O）等内容。

3. 变量、类型、`cout`

3.1 `circle.cpp`

计算给定半径（整数，假设为 `10`）的圆的周长与面积。

要求分别输出半径、周长与面积，每个结果占一行，其中周长与面积保留两位小数。

circle.cpp

```
1  // File: circle.cpp
2
3  #include <iomanip>
4  #include <iostream>
5
6  int main() {
7      int radius{10};
8
9      double circumference{2 * 3.14159 * radius};
```

```

10     double area{3.14159 * radius * radius};
11
12     std::cout << "radius = " << radius << '\n';
13
14     std::cout << std::fixed << std::setprecision(2);
15     std::cout << "circumference = " << circumference << '\n';
16     std::cout << "area = " << area;
17
18     return 0;
19 }

```

预期输出

```

1  radius = 10
2  circumference = 62.83
3  area = 314.16

```

3.1.1 声明/定义语句

Line 7 中的 `int radius{10}` 是声明/定义 (declare/define) 语句，以 `;` 结尾。该语句声明了一个变量 (variable)，其名为 `radius`，其类型 (type) 为 `int` (integer; 整型)，其初始值 (initialization value) 为 `10`。



在声明语句中，`int` 被称为类型说明符 (type specifier)。

如果要将 `radius` 初始化为 `0`，`int radius{0};` 可写成 `int radius{};`

但是，千万不要写 `int radius;` 这样的代码，它声明了一个变量，但没有对其进行初始化。未经初始化的 `radius` 的值是随意的，称为“垃圾值” (garbage)，它们经常对程序造成不易察觉的错误。因此，基本原则是“**始终对声明的变量进行初始化**”。更深入的内容，见 [UB](#) 一节。

我们称 `int radius{10}` 初始化方式为“统一初始化” (uniform initialization，即花括号初始化)。与传统的 `int radius = 10` 初始化方式相比，统一初始化有很多优点，我们推荐始终采用统一初始化方式，这也是 C++ Core Guidelines 中 [ES.23](#) 的要求。



`int radius{10}` 既是声明语句，也是定义语句。并非所有的声明语句都是定义语句，以后会介绍。

3.1.2 变量与类型

C++ 中的每个变量都有类型，在声明语句中指定。类型有两方面含义：

- 第一，类型限定了变量的取值范围。比如，`radius` 是 `int` 类型，则 `radius` 只能取整数值。

- 第二，类型限定了可以施加在变量上的操作集合。比如，可以将 `radius` 与另一个整型变量相加，但是不能将它与一个字符串相加。

对于任何一门程序设计语言来说，类型（或称类型系统）都至关重要。

3.1.3 类型与抽象

这是我们第一次正式接触“抽象”（abstraction）这个概念。所谓抽象，就是隐其细节，究其本质。类型就是一种抽象。比如，对于 `int` 类型，它隐藏掉了计算机内部是如何对整数进行编码、存储的细节，让程序员可以专注在取值范围和允许的操作集合上，更贴近问题本身。

3.1.4 赋值语句

上面提到，在声明变量时，可以对变量进行初始化。之后，我们还可以使用赋值语句修改变量的值，如下所示：

代码块

```
1  int radius{10};  
2  radius = 100;
```

`Line 2` 是一条赋值语句，将 `radius` 的值修改为 `100`。

请注意声明语句与赋值语句的区别：声明语句“创建”了新的变量，需要指明变量的类型，而赋值语句用于修改已有变量的值，其类型信息是隐藏的，不需要显式提供。

3.1.5 运算符、表达式

文件 `circle.cpp` 的 `Line 9`（`Line 10` 类似）定义了名为 `circumference` 的变量，类型为 `double`（双精度浮点数，暂时可以看作数学中的实数），初始值为 `2 * 3.14159 * radius`。其中，`2 * 3.14159 * radius` 是由常量（`2`、`3.14159`）、变量（`radius`）与操作符（operator `*`）组合构成的表达式（expression）。表达式会被求值（evaluated），得出一个结果，该结果作为 `circumference` 的初始值。

C++ 语言中有大量的运算符，我们将陆续在课程中介绍。

3.1.6 `const`

在 `circle.cpp` 中，`Line 9` 与 `Line 10` 中均出现了数字 `3.14159`，这不是一种良好的编程习惯：如果有一天你想将圆周率取为 `3.14`，需要在整个工程中进行查找、替换，极易遗漏、出错。为了避免这种情况，通常的做法是用变量替代 `3.14159` 这种数字（称为 magic number；魔数）：

代码块

```
1  double PI{3.14159};  
2  
3  double circumference{2 * PI * radius};
```

```
4 double area{PI * radius * radius};
```

然而，这种做法仍有缺陷：一旦确定了 `PI` 的取值，我们不希望后续程序修改它。为了表达这种意图，可以使用 `const` 类型限定符（type qualifier）：

代码块

```
1 const double PI{3.14159};
```

`const` 表示 `PI` 是常量，一经初始化则不可被修改。后续代码如果尝试修改它，则会导致编译错误。另外，对于常量，通常使用全大写命名规则。

 准确识别 `const` 的适应场合是优秀程序员的能力之一。

3.1.7 输出： `cout`

Line 12 使用 `std::cout` 输出 `radius` 的值。 `<<` 是插入运算符（insertion operator），作用是把右侧表达式的值“插入”到左侧的输出流 `std::cout` 中。插入运算符可以连续使用，因此 `std::cout << "radius = " << radius << '\n';` 依次输出字符串、变量与换行符。

Line 14 中， `std::fixed` 与 `std::setprecision(2)`（`setprecision` 来自 `<iomanip>` 头文件，参见 [std::fixed / std::setprecision @ cppreference](#)）将浮点数的输出格式设置为定点表示法（fixed-point notation）、保留两位小数。注意：这两个操纵符（manipulator）设置的是输出流（`cout`）的持久状态，设置一次后对后续所有浮点数输出均生效，直到被其他操纵符覆盖。

3.2 sphere.cpp

计算给定半径（整数，假设值为 100）的球体的表面积与体积。

要求每个输出结果占一行，保留 4 位小数，每个输出结果至少占 15 个字符，左对齐。

sphere.cpp

```
1 // File: sphere.cpp
2
3 #include <cmath>
4 #include <iomanip>
5 #include <iostream>
6
7 int main() {
8     const double PI{3.14159};
9
10    int radius{100};
```

```

11
12     double surface_area{4 * PI * std::pow(radius, 2)};
13     double volume{4.0 / 3 * PI * std::pow(radius, 3)};
14
15     std::cout << std::left << std::fixed << std::setprecision(4);
16     std::cout << std::setw(15) << surface_area << " : surface_area\n";
17     std::cout << std::setw(15) << volume << " : volume";
18
19     return 0;
20 }

```

以上输出使用了 `std::left`、`std::fixed`、`std::setprecision`、`std::setw` 操纵符，其中前三个是“粘性”的，会持续影响后续输出。

预期输出

```

1  125663.6000    : surface_area
2  4188786.6667   : volume

```

3.3 `std::format`

`std::format` 是 C++20 标准引入的文本格式化库，定义在 `<format>` 头文件中（参见 [std::format @ cppreference](#)），是现代 C++ 中进行文本格式化的推荐方式。



本节的目的是不是让大家记住各种各样的语法细节，而是了解大致用法，以后遇到时，查手册即可。

3.3.1 `std::format` 基本用法

接下来通过改写以上两段程序，介绍 `std::format` 的基本用法。

3.3.1.1 `circle-format.cpp`

`cout` 需要多次使用 `<<` 拼接输出，复杂格式化冗长笨重。相比之下，`std::format` 将格式字符串与数据分离，更简洁。

```

circle-format.cpp

1  // File: circle-format.cpp
2
3  #include <format>
4  #include <iostream>
5
6  int main() {

```

```

7    int radius{10};
8
9    double circumference{2 * 3.14159 * radius};
10   double area{3.14159 * radius * radius};
11
12   std::cout << std::format("radius = {}\ncircumference = {:.2f}\narea =
    {:.2f}\n", radius, circumference, area);
13   return 0;
14 }

```

格式字符串中的 `{}` 是**替换字段 (replacement field)**，依次对应其后的实参：如 `"circumference = {:.2f}\n"` 中的 `{:.2f}` 对应 `circumference`。

3.3.1.2 sphere-format.cpp

改用 `std::format`，只需一个格式说明 `{:<15.4f}`（左对齐、最小宽度 15、固定 4 位小数），且单次调用生效，无粘性状态：

```

sphere-format.cpp

1  // File: sphere-format.cpp
2
3  #include <cmath>
4  #include <format>
5  #include <iostream>
6
7  int main() {
8      constexpr double PI{3.14159};
9
10     int radius{100};
11
12     double surface_area{4 * PI * std::pow(radius, 2)};
13     double volume{4.0 / 3 * PI * std::pow(radius, 3)};
14
15     std::cout << std::format("{:<15.4f} : surface_area\n{:<15.4f} : volume",
        surface_area, volume);
16
17     return 0;
18 }

```

3.3.2 格式说明 (format specification)

在**替换字段内 (replacement field)** 可以通过格式说明控制输出形式（小数位数、宽度、对齐等）。格式说明的一般形式为 `{:[fill][align][width][.precision][type]}`（参见 [format specification @ cppreference](#)）：

- `width` 指定最小宽度；
- `align` (`<` 左对齐、`>` 右对齐、`^` 居中) 指定对齐方式；
- `.precision` 指定小数位数或有效数字个数；
- `type` 指定类型 (`d` 十进制整数、`f` 固定小数、`e` 科学计数、`g` 通用格式、`c` 字符)。

常用格式说明如下：

格式说明	作用	示例代码	示例输出
<code>{}</code>	默认格式输出	<code>std::format!("{}", 10)</code>	10
<code>{:d}</code>	十进制整数	<code>std::format("{:d}", 42)</code>	42
<code>{:.2f}</code>	浮点数，保留两位小数	<code>std::format("{:.2f}", 3.14159)</code>	3.14
<code>{:<15.4f}</code>	左对齐，最小宽度 15，四位小数	<code>std::format("{:<15.4f}", 3.14159)</code>	3.1416（占 15 字符）
<code>{:.3e}</code>	科学计数法	<code>std::format("{:.3e}", 1.12875e23)</code>	1.129e+23
<code>{:.5g}</code>	通用格式，有效数字 5 位	<code>std::format("{:.5g}", 1.12875e23)</code>	1.1288e+23
<code>{:c}</code>	单个字符	<code>std::format("{:c}", 'm')</code>	m
<code>{{ }}</code>	输出 <code>{ }</code>	<code>std::format("{}")</code>	<code>{ }</code>

非常重要的一点是，`std::format` 是**类型安全 (type-safe)** 的：当格式字符串是字符串字面量时，`std::format` 在编译期检查格式字符串与实参是否匹配：实参不足、类型不匹配都会导致编译失败。即使格式字符串来自运行时变量，不匹配时也会抛出 `std::format_error` 异常。

```
format-error.cpp

1  // File: format-error.cpp
2
3  #include <format>
4  #include <iostream>
5
6  int main() {
7      // (format-error-1)
8      int c_score{59};
9      // 格式字符串中有两个 {}, 但只提供了一个参数: 编译错误
10     // std::cout << std::format("c_score = {} \n c_score = {}", c_score);
11 }
```

```

12 // (format-error-2)
13 int secret{42};
14 // {} 默认按整数输出; 若写成 {:.2f}, 类型不匹配: 编译错误
15 // std::cout << std::format("secret = {:.2f}", secret);
16
17 // (format-error-3)
18 double pi{3.14159};
19 // {:d} 要求整数, 而 pi 是 double: 编译错误
20 // std::cout << std::format("pi = {:d}", pi);
21
22 return 0;
23 }

```

3.4 admin.cpp

输出罗大佑个人信息:

- first_name、last_name、gender (F/M) ;
- birth_year、birth_month、birth_day (mm-dd-yyyy) 、weekday (Xyz.) ;
- cpp_score、music_score、medicine_score;
- mean、standard deviation (sd)。

使用 `std::format` , 格式字符串与数据分离, 每行输出一目了然:

```

admin.cpp
1 // File: admin.cpp
2
3 #include <cmath>
4 #include <format>
5 #include <iostream>
6 #include <string>
7
8 int main() {
9     std::string first_name{"Tayu"};
10    std::string last_name{"Lo"};
11
12    char gender{'M'};
13
14    int birth_year{1954};
15    int birth_month{7};
16    int birth_day{20};
17    std::string weekday{"Tuesday"};
18
19    int c_score{50};
20    int music_score{99};

```

```

21     int medicine_score{78};
22
23     double mean{(c_score + music_score + medicine_score) / 3.0};
24     double sd{std::sqrt((std::pow(c_score - mean, 2) +
25         std::pow(music_score - mean, 2) +
26         std::pow(medicine_score - mean, 2)) / 3.0)};
27
28     std::cout << std::format("{}\t{}\t{}\n", first_name, last_name, gender);
29     std::cout << std::format("{}-{}-{}\t{:.3}\n", birth_month, birth_day,
    birth_year, weekday);
30     std::cout << std::format("{}\t{}\t{}\n", c_score, music_score,
    medicine_score);
31     std::cout << std::format("{:.1f}\t{:.2f}\n", mean, sd);
32
33     return 0;
34 }

```

预期输出

```

1  Tayu\tLo\tM
2  7-20-1954\tTue.
3  50\t99\t78
4  75.7\t20.07

```

weekday 的输出要求为三字母缩写 (Xyz.)。格式说明 `{:.3}` 中的精度 `.3` 对字符串表示只输出前 3 个字符，格式字符串末尾的 `.` 构成缩写 (如 `Tue.`)。

在 C++ 中，使用 `std::string` (C++ 标准库的一部分，通过 `#include <string>` 使用，参见 [std::string @ cppreference](#)) 表示字符串。后续课程会专门介绍 `std::string`。

4. cin

4.1 admin-cin.cpp

```

admin-cin.cpp

1  // File: admin-cin.cpp
2
3  #include <cmath>
4  #include <format>
5  #include <iostream>
6  #include <string>
7
8  int main() {

```

```

9     std::string first_name{};
10    std::string last_name{};
11
12    char gender{};
13
14    int birth_year{};
15    int birth_month{};
16    int birth_day{};
17    std::string weekday{};
18
19    int c_score{};
20    int music_score{};
21    int medicine_score{};
22
23    std::cin >> first_name >> last_name >> gender;
24    std::cin >> birth_year >> birth_month >> birth_day;
25    std::cin >> weekday;
26
27    std::cin >> c_score >> music_score >> medicine_score;
28
29    double mean{(c_score + music_score + medicine_score) / 3.0};
30    double sd{std::sqrt((std::pow(c_score - mean, 2) +
31        std::pow(music_score - mean, 2) +
32        std::pow(medicine_score - mean, 2)) / 3.0)};
33
34    std::cout << std::format("{}\t{}\t{}\n", first_name, last_name, gender);
35    std::cout << std::format("{}-{}-{}\t{:.3}.\n", birth_month, birth_day,
    birth_year, weekday);
36    std::cout << std::format("{}\t{}\t{}\n", c_score, music_score,
    medicine_score);
37    std::cout << std::format("{:.1f}\t{:.2f}\n", mean, sd);
38
39    return 0;
40 }

```

样例输入

```

1  Tayu Lo M
2  1954 7 20
3  Tuesday
4  50 99 78

```

预期输出

```

1  Tayu\tLo\tM
2  7-20-1954\tTue.

```

```
3 50\t99\t78
4 75.7\t20.07
```

`std::cin`（参见 [std::cin @ cppreference](#)）是标准输入流对象。`>>` 是提取运算符（extraction operator），从输入流中读取一个值并存入右侧的变量；可以连续使用，`std::cin >> a >> b` 等价于依次读取两个值。

`std::cin >> x` 按空白符（空格、Tab、换行）分隔输入；读取 `std::string` 时跳过前导空白，读到下一个空白符为止。

4.2 `cin` 用法总结

目标类型	读取的子序列	是否跳过前导空白
<code>int</code>	十进制整数	是
<code>double</code>	浮点数	是
<code>std::string</code>	非空白字符序列	是
<code>char</code>	单个字符	是

 更多规则见 [运算符 >> @ cppreference](#)

若想不跳过空白读取单个字符（例如读取空格或换行本身），可以使用 `std::cin.get(ch)`（参见 [std::istream::get @ cppreference](#)）。

5. UB: Undefined Behavior

5.1 什么是未定义行为？

C++ 标准在 [\[defns.undefined\]](#) 中给出了“未定义行为”的定义：

behavior, upon use of a nonportable or erroneous program construct or of erroneous data, for which this document imposes no requirements.

要理解未定义行为，首先要理解什么是“行为”（behavior）。对此，C++ 标准定义如下：

external appearance or action

程序行为就是程序的外在表现或动作。

未定义行为是指，当使用不可移植或错误的程序构造或者错误数据时，C++ 标准不对其行为施加任何要求的情况。编译器可以随意决定如何解释这些程序，因此不同的编译器可能会产生不同的结果，甚

至在同一个编译器上运行多次也可能产生不同的结果。换句话说，未定义行为会产生完全不可预期的结果。所以，**任何 C++ 程序都要避免包含任何的未定义行为！**

! 建议实际运行本节中的代码，切身感受一下未定义行为带来的惊喜（惊吓）。

下面是初学者**最经常遇到的未定义行为**：

ub-uninitialized-value.cpp

```
1  // File: ub-uninitialized-value.cpp
2
3  #include <iostream>
4
5  int main() {
6      int age;
7      std::cout << "Your age is " << age << "\n"; // use the value of an
        uninitialized variable
8
9      return 0;
10 }
```

因此，良好的编程习惯要求“**始终对变量进行初始化**”。在现代 C++ 中，统一初始化（如 `int age{};`）正是杜绝这种未定义行为的惯用法，这也是 C++ Core Guidelines（[ES.20](#)）的要求。

 以下资料证实使用（局部）变量的未经初始化的值是一种未定义行为（仅供参考，暂时看不懂也没有关系）。

C++ 标准 [dcl.init] 节规定：

If no initializer is specified for an object, the object is default-initialized. When storage for an object with automatic or dynamic storage duration is obtained, the object has an indeterminate value, and if no initialization is performed for the object, that object retains an indeterminate value until that value is replaced.

C++ 标准 [conv.lval] 节进一步规定，读取具有不确定值（indeterminate value）且类型不是 `unsigned char` 的对象，属于未定义行为：

If the glvalue refers to an object whose value is indeterminate and the object has a non-unsigned-char type, the behavior is undefined.

在本学期课程中，我们会反复强调遇到的未定义行为。

5.2 为什么会有未定义行为？

未定义行为的存在并非偶然，主要有以下原因：

- **历史原因**：C 语言诞生于 20 世纪 70 年代，为了追求性能与实现简单，标准刻意不约束某些场景的行为；C++ 继承了这一设计。
- **允许编译器优化**：标准对未定义行为不施加任何要求，意味着编译器可以假设程序中不存在未定义行为，从而进行激进优化。例如，编译器可以假设有符号整数加法不会溢出，并据此简化代码。
- **允许实现差异**：不同硬件平台对某些操作的底层支持与自然行为各不相同，标准对未定义行为不施加统一要求，让各实现按平台实际情况处理，避免为强制统一而付出性能代价。
- **降低标准复杂度**：全面定义所有场景的行为会使标准极其复杂；对错误用法保持“未定义”，让程序员与工具（编译器警告、sanitizer、静态分析器）共同负责。

这也解释了为什么 C++ 社区强调：不要依赖未定义行为的“实际表现”。即使程序在当前编译器上运行正常，更换编译器或开启更高优化级别后，行为也可能改变。

5.3 如何检测未定义行为？

除了人工识别未定义行为之外，还可以借助一些工具：

- 编译器的警告选项：如 `g++` 的 `-Wall -Wextra -Wpedantic`；
- Sanitizer 工具：如 `g++` 的 `-fsanitize=undefined` 检测未定义行为、`-fsanitize=address` 检测内存错误，可通过编译选项开启。

（注：部分内容可能由 AI 生成）